

Exercícios Práticos de Herança

Parte 1: Exercício em Java (Foco em Classes Abstratas e Polimorfismo)

Exercício 1: Classe Abstrata FiguraGeometrica e Classes Concretas Retangulo e Circulo (Nível Intermediário)

O objetivo é criar uma classe base abstrata `FiguraGeometrica` que define um "contrato" comum para todas as figuras e, em seguida, criar classes concretas que herdam e implementam os detalhes específicos de cada figura.

Requisitos:

1. Crie a Classe Base Abstrata `FiguraGeometrica`:

- Defina a classe como `abstract`.
- Adicione um atributo privado:
 - `String nomeDaFigura`
- Crie um construtor que receba o `nomeDaFigura` para inicializar o objeto.
- Implemente um método `get` para o nome (`getNomeDaFigura()`).
- Declare um método **abstrato** público chamado `calcularArea()`, que não terá implementação nesta classe:
 - `public abstract double calcularArea();`
 - Isto força todas as subclasses a fornecerem a sua própria lógica para calcular a área.

2. Crie a Classe Derivada `Retangulo`:

- Faça com que a classe `Retangulo` herde de `FiguraGeometrica`.
- Adicione atributos privados específicos:
 - `double base`
 - `double altura`
- Crie um construtor que receba `base` e `altura`. Este construtor deve chamar o construtor da classe pai (`super`) passando o nome "Retângulo".
- **Implemente** o método `calcularArea()`, sobrescrevendo-o para retornar o cálculo da área do retângulo (`base * altura`).

3. Crie a Classe Derivada `Circulo`:

- Faça com que a classe `Circulo` herde de `FiguraGeometrica`.
- Adicione um atributo privado específico:
 - `double raio`
- Crie um construtor que receba o `raio` e chame o construtor da classe pai (`super`) passando o nome "Círculo".

- **Implemente** o método `calcularArea()`, sobrescrevendo-o para retornar o cálculo da área do círculo (use `Math.PI * raio * raio`).

Parte 2: Exercício em Python (Foco em Reutilização e Sobrescrita de Métodos)

Exercício 1: Classe Personagem e Classes Derivadas Guerreiro e Mago (Nível Básico a Intermediário)

Este exercício explora a herança no contexto de um jogo simples. Criaremos uma classe base `Personagem` com atributos e ações comuns, e classes derivadas que especializam essas ações.

Requisitos:

1. Crie a Classe Base Personagem:

- No construtor (`__init__`), defina os seguintes atributos:
 - `self.nome`
 - `self.nivel`
 - `self.pontos_de_vida`
- Crie um método `exibir_status()` que imprime o nome, nível e pontos de vida do personagem.
- Crie um método `atacar()` que imprime uma mensagem genérica, como `"{self.nome} desfere um ataque básico!"`.

2. Crie a Classe Derivada Guerreiro:

- Faça com que a classe `Guerreiro` herde da classe `Personagem`.
- No construtor (`__init__`), aceite `nome`, `nivel` e `pontos_de_vida`, e adicione um novo atributo:
 - `self.forca` (um valor numérico)
- Lembre-se de chamar o construtor da classe pai (`super().__init__(...)`) para inicializar os atributos herdados.
- **Sobrescreva** o método `atacar()` para que ele imprima uma mensagem específica do guerreiro, como `"{self.nome} ataca com sua espada poderosa!"`.

3. Crie a Classe Derivada Mago:

- Faça com que a classe `Mago` herde da classe `Personagem`.
- No construtor (`__init__`), adicione um novo atributo:
 - `self.magia` (um valor numérico)
- **Sobrescreva** o método `atacar()` para que ele imprima uma mensagem específica do mago, como `"{self.nome} lança um feitiço de fogo!"`.